

ESSENTIAL ASSEMBLY CONCEPTS

1. Registers

Assembly uses registers to store data temporarily for processing:

- **AX, BX, CX, DX:** General purpose 16-bit registers
 - Each can be split into high/low 8-bit parts (AH/AL, BH/BL, etc.)
- **SI, DI:** Source Index and Destination Index registers used for string operations
- **DS:** Data Segment register those points to the data section of the program

2. Memory Organization

The code uses a segmented (meaning with parts) memory model with these sections:

- **.model small:** Defines the memory model
- **.stack 100h:** Allocates space for the stack (used for procedure calls)
- **.data:** Stores all variables and constants
- **.code:** Contains the executable instructions

3. Directives

- **DB (Define Byte):** Allocates memory for strings or single bytes (db "Hello", "\$")
- **DW (Define Word):** Allocates memory for 16-bit values (addresses)
- **DUP:** Used to duplicate a value multiple time (10 dup('-'))

4. Instructions

- **MOV:** Copies data between registers or memory (mov ax, bx)
- **CMP:** Compares two values and sets flags (cmp al, '\$')
- **JMP:** Unconditional jump to a label (jmp gameloop)
- **JE, JNE, etc.:** Conditional jumps based on comparison results
- **INT:** Calls system interrupts for I/O operations
- **CALL:** Executes a procedure/function
- **RET:** Returns from a procedure
- **PUSH/POP:** Saves/retrieves values from stack

- **INC/DEC:** Increments/decrements values
- **ADD/SUB:** Arithmetic operations

5. Addressing Modes

- **Immediate:** Direct value (mov al, 10)
- **Register:** Using register value (mov ax, bx)
- **Direct:** Using memory address (mov al, [si])
- **Indirect:** Using register as pointer (mov si, offset word)

6. Interrupts

DOS and BIOS services used in this code:

- **INT 21h:** DOS functions
 - AH=01h: Read character with echo
 - AH=02h: Display character
 - AH=09h: Display string (terminated with '\$')
- **INT 10h:** Video services

7. PROCEDURES AND MARCOS

MACROS DEFINITION:	PROCEDURE DEFINITION:
In assembly language, a macro is like a template or shortcut that lets you define a group of instructions once and then reuse it wherever you need.	Like Macros A procedure (also called a subroutine or function) is; a block of code that performs a specific task and can be called from multiple places in the program. However, they are slightly different as particularly used for larger design modules
<pre>.model small .stack 100h .data msg1 db "Hello, World!", "\$" msg2 db "Assembly is cool!", "\$" .code main: mov ax, @data mov ds, ax</pre>	<pre>.model small .stack 100h .data prompt db "Enter a character: \$" newline db 0Dh,0Ah, "\$" message db "You entered: \$" .code main: mov ax, @data</pre>

<pre> ; Define macro to print a string PRINT MACRO str_addr mov ah, 09h ; DOS print string function lea dx, str_addr ; Load address of string into DX int 21h ; Call DOS interrupt ENDM ; --- Using the macro --- PRINT msg1 ; This will print: Hello, World! mov ah, 09h mov dx, offset newline int 21h PRINT msg2 ; This will print: Assembly is cool! ; Exit program mov ah, 4Ch int 21h newline db 0Dh,0Ah, "\$" ; Just a new line end main </pre>	<pre> mov ds, ax ; Ask for input lea dx, prompt call print_string ; Get character from user mov ah, 01h int 21h mov bl, al ; Save character in BL ; Print new line lea dx, newline call print_string ; Print message lea dx, message call print_string ; Display the character entered mov dl, bl ; Move character to DL for printing mov ah, 02h int 21h ; Print new line lea dx, newline call print_string ; Exit program mov ah, 4Ch int 21h ; ----- ; PROCEDURE: print_string ; Description: Prints a string whose address is in DX ; ----- print_string PROC mov ah, 09h int 21h ret print_string ENDP end main </pre>
--	---

GAME FLOW

1. Display title and welcome message
2. Get difficulty level input (e/m/h)
3. Randomly select a word based on difficulty
4. Initialize game state (guesses word with dashes)
5. Main game loop:
 - Display hangman ASCII art based on chances left
 - Show chances and score
 - Get letter input from user
 - Check if the letter was already entered
 - Compare letter with word and update guessed word
 - Update game state (score, chances)
 - Check win/lose conditions
6. Display result (win/lose)

Key Procedures

- **get_string**: Select random word based on difficulty
- **is_entered**: Check if letter was already guessed
- **compare_and_store_word**: Check letter against word
- **end_game**: Update game state and check win condition

Game Logic

- Word is selected randomly from arrays based on difficulty
- Player guesses one letter at a time
- Correct guesses reveal letter positions and add points
- Wrong guesses reduce chances and show more of hangman
- Game ends when word is guessed or chances run out